Solving General Twisty Puzzles with RL

Twisty puzzles = Rubik's cube, cuboid generalisations, geared variations and more

Goal: Compute interpretable solutions on consumer hardware

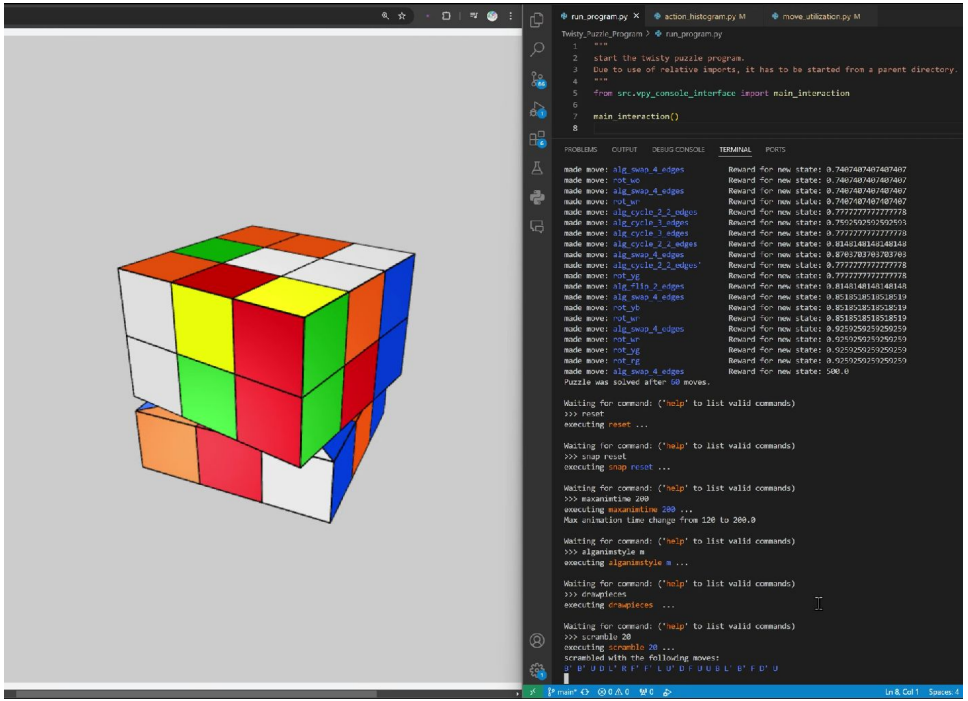
Methods:

- Automatic action space augmentation
- Dense, intuitive rewards

Result:

- General method matching sample-efficiency of human reward-engineered solutions
- Solved Rubik's Cube with ~10^{19} states in 10^{8} training steps.
- AI solutions similar to human ones (e.g. corners -> edges)

Time: ~600 hours incl. thesis



UI showing Rubik's Cube solution in progress

[video explanation]

[Code on GitHub] [master's thesis] [more demo videos]

Aug 2020 - Dec 2024

Custom Ticket to Ride Map Creation Tools

Goal: Efficiently create custom Ticket to Ride versions

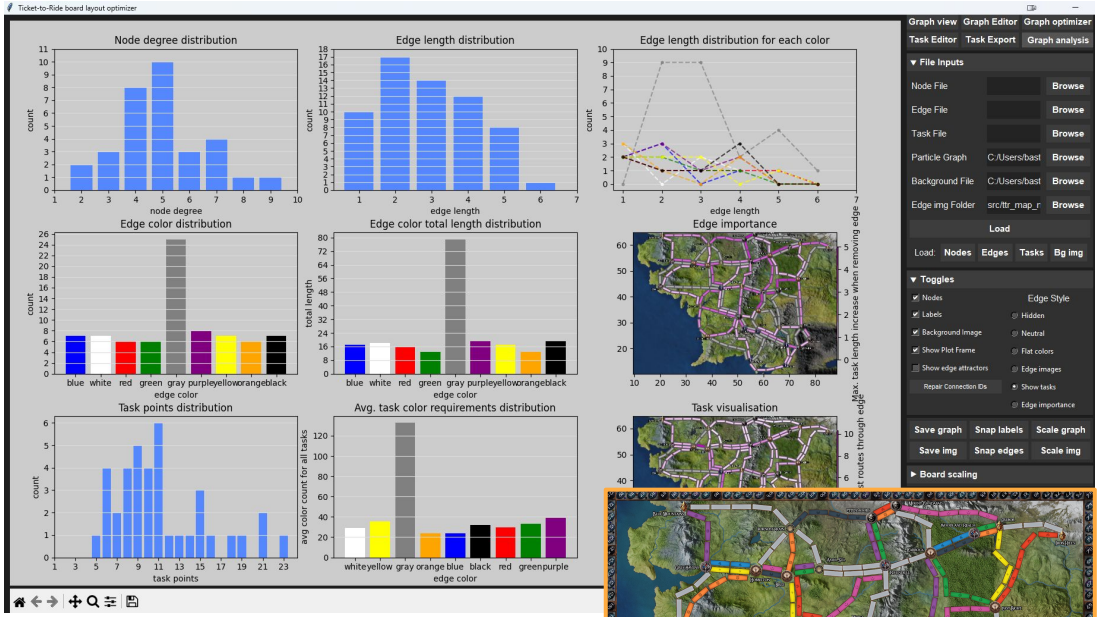
Methods:

- Analyze creation processes and bottlenecks
- Build tools to support creation at multiple stages

Result:

- Faster creation of variants
- High-quality custom games

Time: ~300 hours



Graph analysis view and final map output

[Code on GitHub](#) [result showcase](#)

Dec 2022 - Dec 2023

Simulating real-world traffic sensor noise for more robust RL-Traffic Signal Control

Goal: Assess robustness of RL-TSC agents against noisy data

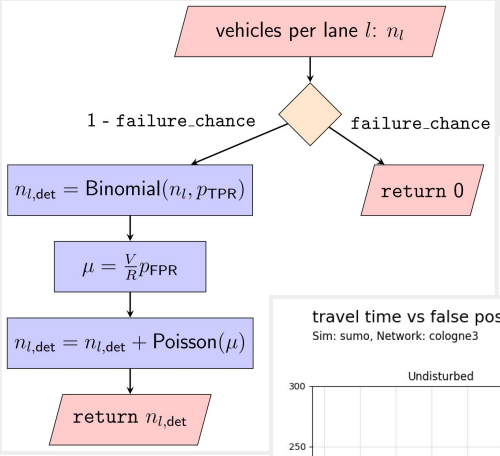
Methods:

- Simulate realistic sensor noise
- >600,000 simulated hours to test parameter settings and analyse their impact

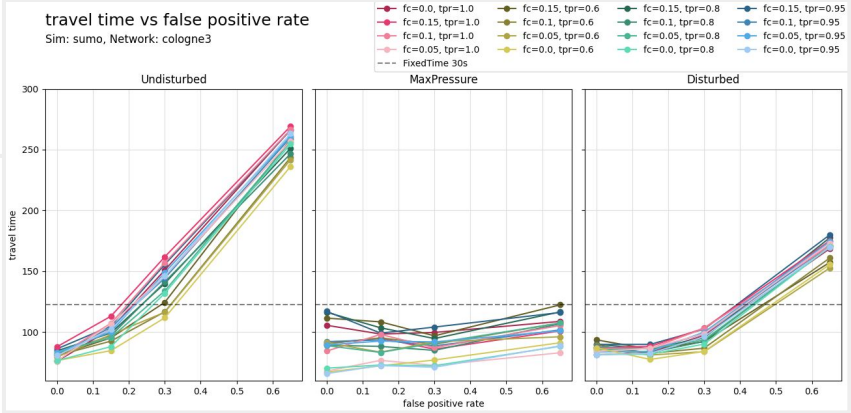
Result:

- Sensor noise affects RL agents much more than previous methods.
- Noise model tuneable for different real world traffic sensors

Time: ~200 hours incl. writing paper



Simulating sensor noise
Noisy calculation of detected vehicles from actual vehicles



[Code on GitHub]

publication planned

Oct 2023 - Aug 2025

Northgard Board Game with Web-UI

Northgard combines the three games Risk, Carcassonne and Dominion

Goals:

- Recreate the board game Northgard digitally
- Write web-UI without prior JS/CSS/html knowledge

Methods:

- AI tools for writing frontend
- Python backend
- Custom tools for asset creation

Result:

- Prototype with responsive web-UI

Time: ~100 hours incl. asset creation

[Code on GitHub](#) [demo video](#) [game info](#)

[Assets on GitHub](#)

Dec 2024 - Sep 2025

Wizard Game with multiple AI players

Goal: Compare different AI techniques for creating wizard game agents

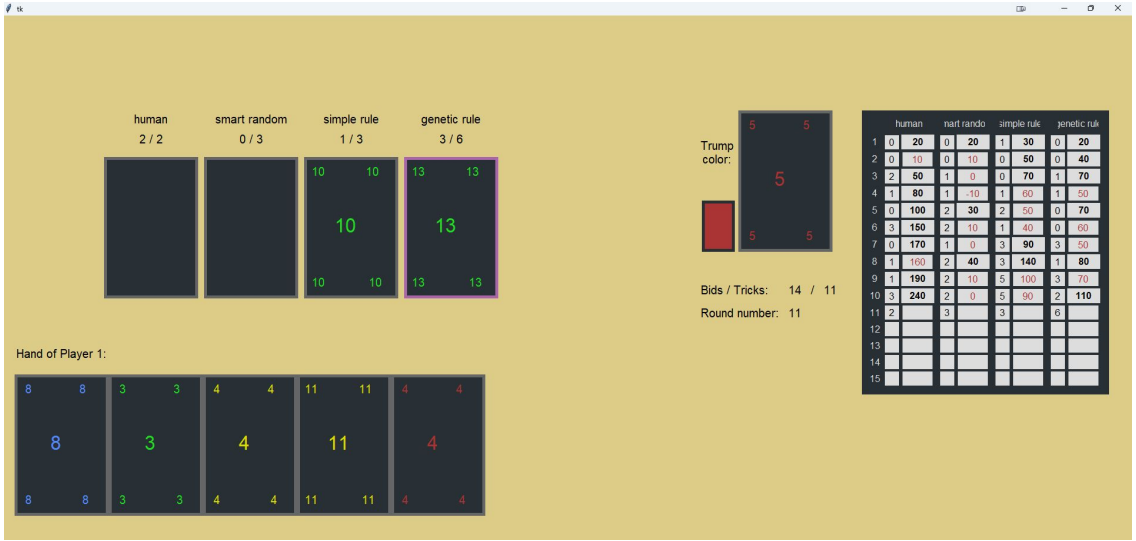
Methods:

- Rule-based AI, optimized via genetic evolution
- tkinter GUI design

Result:

- Major capability differences between AI agents

Time: ~60 hours



[Code on GitHub]

Apr 2023

Simplified Saboteur Game with RL players

Goal: Investigate emergence of deceptive AI behaviour

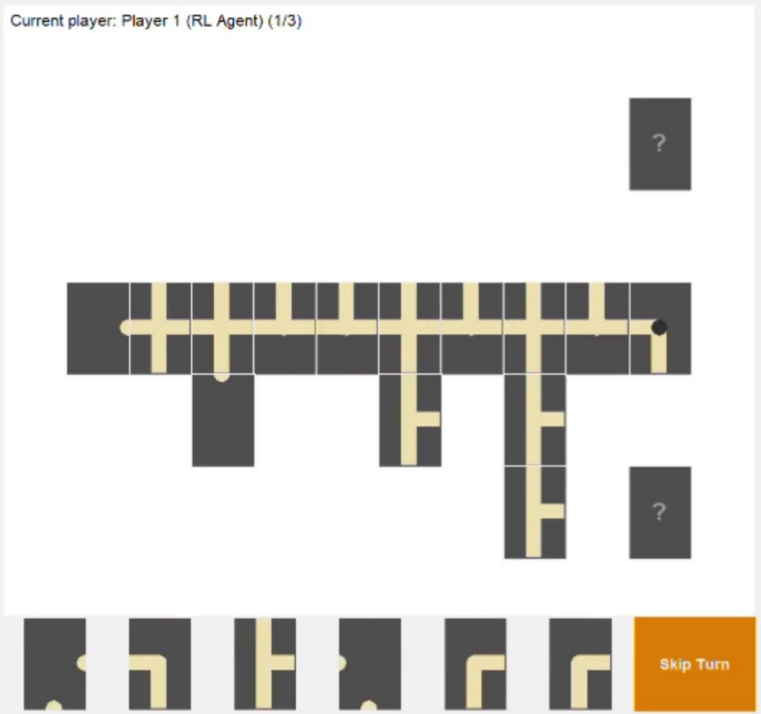
Methods:

- LLM-accelerated coding

Result:

- RL-agents successfully play simplified game
- AI strategies similar to human ones.

Time: ~20 hours



Code on GitHub

demo video

Maze Generator & Solver using Q-Learning

Goals:

- Automatic maze generation
- Q-learning based solver

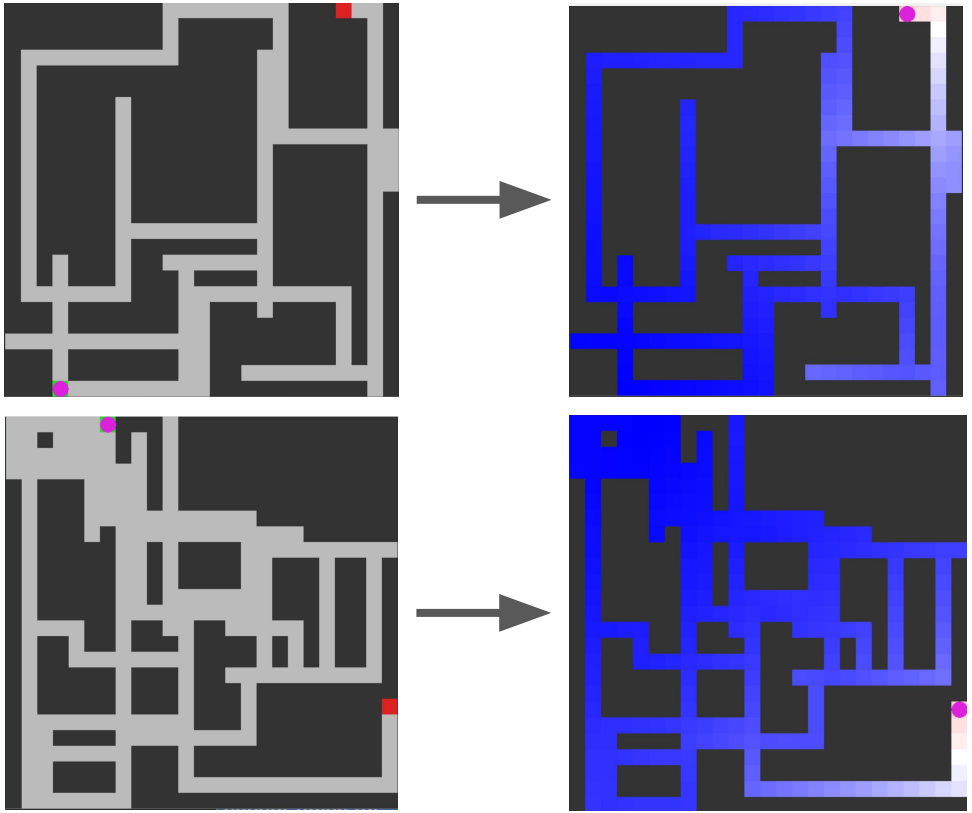
Methods:

- Customized random walk
- Q-learning
- tkinter GUI design

Result:

- Reliable, efficient solver for small mazes

Time: 3 days ≈ 20 hours



Apr 2021

Learning Methods in Humans, Animals and AI

Goal: Compare learning techniques in different organisms to inspire future research

Methods:

- Offered Workshop at VANconference25
- Years of following current research

Result:

- Overview helps predict industry & research direction

Time: <10 hours

Learning method	Animals	Humans	Computer (Machine Learning)
Evolution	genetic evolution	genetic evolution	genetic algorithms (e.g. NEAT)
Trial & Error	Conditioning	Conditioning	Reinforcement learning
Imitation	Orcas teaching hunting methods & language	Learning language	Supervised learning, Imitation learning
Abstract Instructions	Bee dance to describe location of food	Most of school / higher education, baking recipes	In-context learning in LLMs
Mental Simulation	Birds cracking nuts using cars, tool use	Dreams, pondering other's reactions	World models (e.g. DreamerV3)

GitHub page

Local LLM conversation viewer

Goal: Local GUI to view LLM transcripts from OpenAI, Anthropic and LM Studio

Methods:

- Vibe-Coding with GitHub CoPilot agent

Result:

- Modern web-UI to view chats from different sources.

Time: <5 hours

[image coming soon]

Code on GitHub